

Урок 13 — ФИНАЛ: боевая прошивка машинки

Проектный урок · 90 минут · нужны все предыдущие уроки

День сборки

Сегодня всё, что вы учили тринадцать уроков, соберётся в одну программу — полную боевую прошивку вашей машинки: три ловушки по кнопкам из телефона плюс приём попаданий. С этой прошивкой машинка выходит на гонку.

Переходим во вкладку «Компилятор»

Финальный проект пишем не в уроке, а во вкладке «Компилятор». Откройте её — и вы увидите, что часть строк уже написана и отмечена замками. Это не случайно.

Присмотритесь к защищённым строкам: это блок `proverkaPopadaniya()` — приём ИК-выстрела (приёмник на D11) и сигнал малинке «в нас попали» (импульс на D5). По правилам гонок ЛЮБАЯ машинка на трассе обязана глохнуть при попадании — даже если её хозяин решил не ставить свою пушку. Поэтому этот блок вшит в каждый проект намертво: его нельзя ни изменить, ни удалить. Всё остальное — ваша территория: дополняйте код в свободных местах.

[МЕСТО ДЛЯ ИЛЛЮСТРАЦИИ] Скриншот вкладки «Компилятор»: шаблон с замками на блоке попадания, стрелки к свободным зонам «твой код здесь». Рядом — судья гонок со свистком: «Попал — стой!».

Проверим провода

Карта из Урока 2, финальная сверка: D2 ← кнопка «1» (сбросник), D3 ← кнопка «2» (стробоскоп), D4 ← кнопка «3» (ИК-выстрел), D5 → малинке «попадание», D6 → лента, D9 → серво, D10 → ИК-светодиод, D11 ← ИК-приёмник. Питание модулей — от силового распределителя, все клеммы затянуты.

Собираем прошивку

Добавляйте блоки в РЕДАКТИРУЕМЫЕ зоны шаблона. Заметьте: `IRremote` подключать не нужно — он уже в защищённой части.

В зону над `setup` (после строки с `#include`):

```
#include <FastLED.h>
#include <Servo.h>

#define LENTA_PIN 6
#define KOL_LED 8
```

```
CRGB leds[KOL_LED];
Servo servo;
int UGOL_ZAKRYT = 0;
int UGOL_OTKRYT = 90;
```

В setup(), в свободную зону:

```
FastLED.addLeds<WS2812B, LENTA_PIN, GRB>(leds, KOL_LED);
FastLED.setBrightness(60);
servo.attach(9);
servo.write(UGOL_ZAKRYT);
pinMode(2, INPUT);
pinMode(3, INPUT);
pinMode(4, INPUT);
IrSender.begin(10);
```

В loop(), в свободную зону (проверка попадания уже вызывается выше — защищённой строкой):

```
if (digitalRead(2) == HIGH) {
    sbros();
}
if (digitalRead(3) == HIGH) {
    strob();
} else {
    zalit(CRGB::Black);
}
if (digitalRead(4) == HIGH) {
    vystrel();
}
```

В зону «свои функции» (внизу, перед обязательным блоком):

```
void zalit(CRGB cvet) {
    for (int i = 0; i < KOL_LED; i++) {
        leds[i] = cvet;
    }
    FastLED.show();
}

void strob() {
    zalit(CRGB::White);
    delay(40);
    zalit(CRGB::Black);
    delay(60);
}

void sbros() {
    Serial.println("Sbros!");
    servo.write(UGOL_OTKRYT);
}
```

```
    delay(700);  
    servo.write(UGOL_ZAKRYT);  
    delay(1000);  
}  
  
void vystrel() {  
    IrSender.sendNEC(0x01, 0x59, 0);  
    Serial.println("Pyu!");  
    delay(500);  
}
```

Испытания

Нажмите «Проверить» — программа большая, читайте ошибки внимательно и чините по одной. Затем загрузите на плату и проведите полный тест-драйв по чеклисту: 1) кнопка «1» — сбросник роняет груз и закрывается; 2) кнопка «2» — лента слепит, отпустили — гаснет; 3) кнопка «3» — камера телефона видит вспышки ИК; 4) «выстрел» пультом в приёмник — в мониторе «POPADANIE!», малинка глушит мотор. Четыре галочки? Машинка готова к гонке!

Замечание инженера: пока выполняется `delay` (например, перезарядка сбросника), программа «спит» и не проверяет остальные кнопки — на гонке это доли секунды, но вы это заметите. Устранение таких пауз (магия `millis`) — тема следующего уровня курса. Дорасти до неё — отличная цель!

★ Задания на прокачку

Задание 1. Фирменный стиль: замените белый стробоскоп на свой боевой цвет и добавьте «полицейский» вариант со второй половиной ленты.

Задание 2 ★. Свето-сигнал попадания: в СВОЁМ коде `loop` следите за пином 5?.. Нет — проще: добавьте в свободную зону `loop` проверку и вспышку красным, когда сработал сброс или выстрел — чтобы зрители видели действия машинки.

Задание 3 ★★. Придумайте и реализуйте СВОЮ фишку для гонки: двойной выстрел, «дымовая завеса» бегущим огнём, сброс с задержкой-сюрпризом... Самая зрелищная фишка получает место в общем зачёте!

Мини-тест

Ниже — мини-тест по уроку. В приложении Smart Bolashaq он превращается в интерактивные карточки с проверкой ответов; в обычной PDF-читалке этот блок выглядит как мелкие служебные данные — так и задумано.

```
SBTEST1|k=1,3,2,2,2
?Почему_блок_proverkaPopadaniya()_защищён_замками?
*Это_правило_гонок:_ЛЮБАЯ_машинка_обязана_глохнуть_при_попадании
*Он_самый_красивый
*Его_написал_директор
!Даже_если_хозяин_машинки_не_ставит_свою_пушку,_принимать_попадания_он_обязан_—_поэтому_
~блок_нельзя_изменить_или_удалить_.
?Где_пишется_финальная_боевая_прошивка?
*В_отдельном_текстовом_файле
*В_уроке,_как_раньше
*Во_вкладке_«Компилятор»_с_шаблоном
!Уроки_—_тренировки_с_чистым_редактором._Боевая_прошивка_собирается_в_«Компиляторе»,_где
~_вшит_обязательный_блок_.
?Что_можно_делать_с_защищёнными_строками_шаблона?
*Изменять_их
*Ничего_—_только_дописывать_СВОЙ_код_в_свободных_зонах
*Удалять_их
!Замочные_строки_неприкосновенны._Ваша_территория_—_свободные_зоны:_над_setup,_внутри_se
~tur_и_loop,_зона_своих_функций_.
?Нужно_ли_самому_писать_#include_<IRremote.h>_в_финальном_проекте?
*Да,_обязательно
*Нет_—_он_уже_есть_в_защищённой_части_шаблона
*IRremote_вообще_не_нужен
!Шаблон_уже_подключает_IRremote_и_вызывает_proverkaPopadaniya()_в_loop._Ваши_#include_—
~только_FastLED_и_Servo_.
?Что_входит_в_полный_тест-драйв_перед_гонкой?
*Только_проверка_мотора
*Проверка_всех_трёх_ловушек_и_приёма_попадания
*Мойка_машинки
!Чеклист:_сбросник_по_кнопке_«1»,_стробоскоп_по_«2»,_ИК-выстрел_по_«3»_и_«POPADANIE!»_пр
~и_выстреле_в_приёмник.
```